Progress Report 2

Project Title

AI-Based Autonomous Drone Navigation in a Simulated 3D Urban Environment

Course

COMP 569 Artificial Intelligence

1. Scope Update and Implementation Overview

After the feedback on Progress Report 1, the project scope was narrowed to focus only on autonomous drone navigation. The workforce scheduling and CSP portion has been removed completely. This change made the project more coherent and allowed development effort to concentrate on one well-defined AI problem: optimal path planning for a drone in a 3D urban environment. It also directly addressed the instructor's question about how the drone environment would be simulated.

The current prototype consists of three connected parts:

1. A Python search engine that models the navigation problem as a 3D state-space search problem.
2. A FastAPI backend that receives an environment definition and returns an optimal path with evaluation metrics.
3. A React and TypeScript frontend that simulates the urban environment visually using `react-three-fiber` and animates the resulting drone path.

The system currently supports:

- A discrete 3D grid world with states represented as `s = (x, y, z)`.
- Start and goal coordinates.
- Obstacles and no-fly zones treated as impassable states.
- Uniform Cost Search (UCS) as a baseline algorithm.

- A* search with Manhattan, Euclidean, and a prototype building-aware heuristic.
- A visual 3D simulation showing the city, route, and drone animation.
- Output metrics including total cost, nodes expanded, and runtime.

The simulation approach answers the question raised in the instructor feedback. Instead of using Unity, the prototype currently uses a web-based 3D simulation layer built with React, Three.js, and `react-three-fiber`. The frontend sends a pathfinding request to the FastAPI backend and then animates the returned path inside a 3D urban grid. This decision kept the integration simpler while still providing a clear and convincing simulation of drone movement through a constrained environment.

Technology Stack

- Backend: Python, FastAPI, Pydantic
- Frontend: React, TypeScript, Vite
- 3D Visualization: Three.js, `@react-three/fiber`, `@react-three/drei`
- API Communication: Axios

System Functionality

At a high level, the workflow is as follows:

1. The user defines or generates an urban grid environment.
2. The frontend sends the environment description to the `/pathfind` FastAPI endpoint.
3. The backend constructs the environment and runs either UCS or A*.
4. The backend returns the selected path, total path cost, nodes expanded, and runtime.
5. The frontend renders the path as a line and animates the drone along the returned coordinates.

This means the project has already moved beyond planning and design. A working prototype exists, and the student can now analyze behavior, compare algorithms, and identify remaining weaknesses before final submission.

2. Algorithm Implementation

2.1 Problem Representation

The navigation problem is implemented using the state-space formulation:

`<S, A, T, C, G>`

where:

- `S` is the set of valid drone states in the 3D grid.
- `A` is the set of available movement actions.
- `T` is the transition function that maps one state to a neighboring state.
- `C` is the movement cost function.
- `G` is the goal state.

Each state is represented as:

`s = (x, y, z)`

where `x` and `y` are horizontal coordinates and `z` is altitude.

The environment takes the following inputs:

- Grid bounds `(max_x, max_y, max_z)`
- Start coordinate
- Goal coordinate
- Obstacles
- No-fly zones

Both obstacles and no-fly zones are stored as impassable cells, which means the search algorithm never expands them.

2.2 Transition and Cost Model

The transition model supports six local actions:

- Move horizontally in `+x`
- Move horizontally in `-x`

- Move horizontally in `+y`
- Move horizontally in `-y`
- Move upward in `+z`
- Move downward in `-z`

The cost model is intentionally asymmetric to simulate energy usage:

- Horizontal movement = 1
- Downward movement = 1
- Upward movement = 2

This is one of the most important design choices in the project because it makes the problem more realistic than a simple shortest-path search. The drone is encouraged to avoid unnecessary climbing, which creates a meaningful difference between geometric distance and energy cost.

2.3 Uniform Cost Search

UCS is implemented as the baseline algorithm. The search engine uses a priority queue ordered by cumulative path cost `g(n)`. For each node expansion:

1. The state with the lowest known path cost is popped from the queue.
2. Valid neighbors are generated.
3. New path costs are computed.
4. A neighbor is updated only if a cheaper path to that state has been found.

Because all movement costs are non-negative, UCS guarantees an optimal solution. However, as expected, its main weakness is the number of nodes that must be expanded before reaching the goal.

2.4 A* Search

The A* implementation uses the same priority-queue structure but orders states by:

```
f(n) = g(n) + h(n)
```

where `g(n)` is the accumulated cost so far and `h(n)` is a heuristic estimate of the remaining cost.

Three heuristics are currently implemented:

Manhattan Heuristic

```
h(n) = |x - x_g| + |y - y_g| + |z - z_g|
```

This heuristic is simple and computationally cheap. Even though it does not explicitly weight upward movement by 2, it still acts as a lower bound and therefore works well as a conservative guide for A*.

Euclidean Heuristic

```
h(n) = sqrt((x - x_g)^2 + (y - y_g)^2 + (z - z_g)^2)
```

This heuristic captures straight-line distance in 3D space. It is also admissible in this discrete setting because it remains below the true stepwise travel cost.

Prototype Building-Aware Heuristic

```
h(n) = d_xy + max(0, h_obs - z)
```

This heuristic attempts to account for obstacle height by encouraging the search to reason about vertical clearance. The goal was to make the heuristic more realistic for an urban environment where buildings matter, not just Euclidean distance.

2.5 Key Design Decisions

Several design choices shaped the implementation:

- The environment model is separate from the API layer, which keeps the search engine reusable.
- The backend returns both the path and evaluation metrics, so the visualizer can serve analytical as well as demonstrative purposes.
- The frontend and backend communicate through a simple JSON contract, making the simulation architecture easy to extend.
- The current visualization stack uses React and Three.js rather than Unity. This is a deviation from the earlier idea, but it still fulfills the simulation requirement and reduced development overhead.

2.6 Deviations From the Original Plan

There were two significant deviations from the original idea presented in Progress Report 1:

1. Workforce scheduling was removed completely in response to scope feedback.
2. The simulation visualizer was implemented with React and `react-three-fiber` instead of Unity.

Both changes improved feasibility. The project is now narrower, more technically consistent, and already demonstrable through a working prototype.

3. Preliminary Results

3.1 Validation Approach

The current prototype was validated in three ways:

1. Direct Python execution of the search engine on controlled scenarios.
2. End-to-end API validation through the FastAPI `/pathfind` endpoint.
3. Frontend production build verification to confirm the React visualizer compiles successfully.

An end-to-end API request using FastAPI's test client returned HTTP 200 and a valid path from start to goal in the sample urban map. During report preparation, small TypeScript cleanup changes were made so the frontend now builds successfully with `npm run build`.

3.2 Sample API Output

The following sample case was tested:

- Bounds: `(10, 10, 10)`
- Start: `(0, 0, 0)`
- Goal: `(9, 9, 9)`
- Obstacles: one building column at `(5, 5, z)` for `z = 0..5`
- No-fly zones: none

Sample result:

- Status code: `200`
- Path found: yes
- Cost: `36`
- Nodes expanded with A* Manhattan: `1158`
- Path length: `28` coordinates

The returned path correctly routed around the obstacle column and then climbed to the goal altitude.

3.3 Benchmark Scenarios

To generate preliminary evidence, three benchmark scenarios were executed repeatedly and the mean values were recorded. The results below summarize the current behavior of the prototype.

Scenario A: Open Grid

Environment:

- Bounds: `(8, 8, 6)`
- Start: `(0, 0, 0)`
- Goal: `(7, 7, 4)`
- Obstacles: none

Algorithm	Mean Cost	Mean Nodes Expanded	Mean Runtime (ms)
UCS	22.0	530.0	13.704
A* Manhattan	22.0	368.0	6.950
A* Euclidean	22.0	405.0	13.053
A* Building-Aware	22.0	386.0	9.807

Observation: In an open environment, all algorithms found the same optimal cost, but A* Manhattan reduced node expansions by about 30.6% compared with UCS.

Scenario B: Single Building Column

Environment:

- Bounds: `(10, 10, 10)`
- Start: `(0, 0, 0)`
- Goal: `(9, 9, 9)`
- Obstacles: building column at `(5, 5, z)` for `z = 0..5`

Algorithm	Mean Cost	Mean Nodes Expanded	Mean Runtime (ms)
UCS	36.0	1309.0	33.487
A* Manhattan	36.0	1158.0	30.433
A* Euclidean	36.0	1224.0	33.415
A* Building-Aware	36.0	1180.0	26.721

Observation: The search problem becomes more expensive as altitude changes and obstacles interact. A* still improves over UCS, but the improvement is more modest because the goal itself is at a high altitude.

Scenario C: Dense Urban Blocks

Environment:

- Bounds: `(10, 10, 8)`
- Start: `(0, 0, 0)`
- Goal: `(9, 9, 5)`
- Obstacles: multiple multi-cell building blocks
- No-fly zones: one restricted vertical strip

Algorithm	Mean Cost	Mean Nodes Expanded	Mean Runtime (ms)
UCS	28.0	981.0	19.973
A* Manhattan	28.0	641.0	15.103
A* Euclidean	28.0	721.0	10.819

Algorithm	Mean Cost	Mean Nodes Expanded	Mean Runtime (ms)
A* Building-Aware	28.0	641.0	16.495

Observation: This scenario is the most representative of the intended urban use case. A* Manhattan reduced node expansions by about 34.7% relative to UCS, while preserving the same optimal cost. In this denser environment, the building-aware heuristic matched Manhattan in nodes expanded, which suggests that obstacle-sensitive guidance can be useful, although its theoretical validity still needs work.

3.4 Interpretation of Results

The preliminary results are consistent with expectations from AI search theory:

- UCS always found an optimal path, but at the cost of more node expansions.
- A* was consistently more efficient than UCS in the tested scenarios.
- Manhattan generally provided the best balance between simplicity and efficiency.
- Euclidean also improved performance, but usually not as much as Manhattan.
- The current building-aware heuristic can guide the search effectively in some dense maps, but it is not yet ready to be treated as a fully validated admissible heuristic.

These results show meaningful progress. The project has moved beyond a conceptual design and already demonstrates measurable algorithmic behavior in a realistic prototype setting.

4. Challenges and Limitations

4.1 Scope Management

The first major challenge was scope. Progress Report 1 combined informed search and workforce scheduling, which made the project too broad. After feedback, the project was narrowed to a single domain problem: drone navigation in a simulated urban environment. This adjustment improved feasibility and gave the project a clearer technical identity.

4.2 Heuristic Validation

The most important technical limitation discovered during this stage concerns the current building-aware heuristic. Although it was designed to reflect obstacle height, additional checking revealed that the present formula can overestimate the true remaining cost in some maps.

One counterexample produced:

- Building-aware heuristic value: `16`
- True optimal remaining cost from UCS: `12`

This means the current implementation is not always admissible, even though admissibility was one of the intended design goals. This is an important finding rather than a failure, because it identifies exactly where further refinement is needed before the final submission. For that reason, the strongest current baseline for formal comparison is A* with Manhattan and Euclidean heuristics.

4.3 Simulation Limitations

The current environment is a discrete and deterministic grid world. That is appropriate for the course focus on search, but it still simplifies real drone navigation in several ways:

- No wind, weather, or uncertainty is modeled.
- No battery depletion model exists beyond the asymmetric movement cost function.
- Obstacles and no-fly zones are static.
- The frontend currently visualizes no-fly zones, but the UI does not yet provide a rich editor for them.

These limitations are acceptable for a course prototype, but they should be acknowledged clearly.

4.4 Performance and Engineering Limitations

The frontend now compiles successfully, but there are still practical engineering issues to refine:

- The production bundle is relatively large because 3D rendering libraries are heavy.
- Runtime measurements are small enough that they can vary between runs, so larger benchmark sets are needed for stronger analysis.
- The current test coverage is still light and should be expanded before final submission.

5. Plan for Final Submission

The project is now in a solid prototype stage, but several steps remain before the final submission is complete.

5.1 Remaining Tasks

1. Replace or revise the building-aware heuristic so that it is provably admissible and consistent, or clearly justify a different validated heuristic if that is the better final choice.
2. Expand the benchmark suite with more map densities, more no-fly-zone cases, and repeated trials for cleaner comparisons.
3. Improve the frontend controls so the user can define no-fly zones and more complex city layouts directly from the interface.
4. Capture final screenshots and polish the visual presentation of the simulation.
5. Add stronger automated testing for both the search engine and API behavior.
6. Prepare the final written analysis, including charts or plots that summarize node expansion and runtime trends.

5.2 Working Timeline

Working timeline beginning from April 3, 2026:

- April 4-6, 2026: revise heuristic design and verify admissibility with targeted test cases
- April 7-9, 2026: expand benchmark scenarios and collect cleaner comparison data

- April 10-12, 2026: improve frontend controls, simulation presentation, and screenshots
- April 13-15, 2026: finalize testing, write the final report, and prepare the presentation/demo

5.3 Planned Improvements

The final submission should improve both the AI analysis and the presentation quality. The main improvements planned are:

- stronger heuristic justification
- more rigorous benchmarking
- a more polished interactive simulation
- clearer evidence that the final heuristic choice is correct and efficient

6. Conclusion

This project has made meaningful technical progress since Progress Report 1. The scope has been narrowed appropriately, the drone navigation engine has been implemented, the FastAPI backend is working, and a 3D frontend simulation already demonstrates the pathfinding process visually. Preliminary results show that A* improves efficiency over UCS while preserving optimal path cost in the tested environments.

At the same time, this stage also revealed an important limitation: the current building-aware heuristic is promising in practice but not yet theoretically safe in all cases. Identifying that issue now is useful because it gives the project a clear and academically responsible direction for the final stage. Overall, the system is no longer just a proposal; it is a functional prototype with measurable results and a well-defined path to completion.